AMD Unsloth GRPO Fine-tuning

Malini Bhandaru | Shailen Sobhee

AMD
together we advance_

Notebook access

notebooks.amd.com/ind-dev

Password: RadeonUnslothTuning

Agenda

01 **Fine-tuning: what and when**
The problem it solves

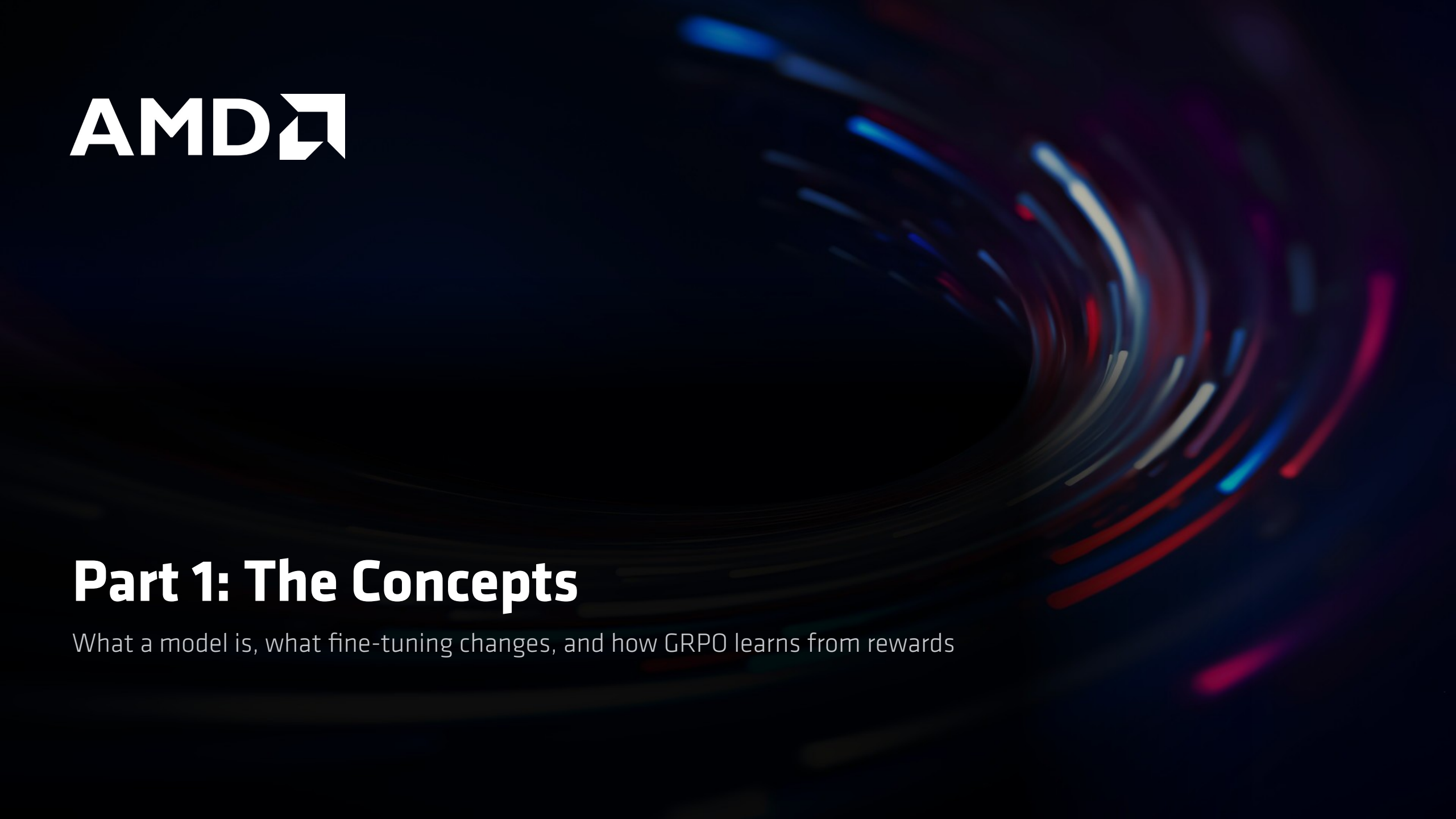
02 **Types of fine-tuning**
LoRA, QLoRA, SFT, GRPO

03 **The Unsloth advantage**
Faster, leaner, no-code

04 **A game of Sudoku**
Our reward-driven task

05 **AMD Radeon GPU**
rocm-smi and rocminfo

06 **Hands-on notebook**
Run it yourself



Part 1: The Concepts

What a model is, what fine-tuning changes, and how GRPO learns from rewards

What is a Large Language Model?

A neural network trained on a huge amount of text so it can predict **the next token** given what came before.

That one simple skill, done at enormous scale, is enough to chat, summarise, write code and reason.

Base models

Raw next-token predictors. Great starting point for training, awkward to chat with.

Instruct models

Further trained to follow instructions and hold a conversation.

Generalists

Llama, Qwen, Mistral on Hugging Face know a bit of everything, your job in particular: no.

What is fine-tuning? Why do it?

Take a pre-trained model and keep training it a little more on **YOUR** data, so it adapts to **YOUR** task, domain, tone or format.

1

Voice

Speak in a specific style or persona: your brand, your support agent.

2

Domain

Know legal, medical, or your own product documentation.

3

Format

Always return JSON. Always cite sources. Every time.

4

Focus

Do one narrow task extremely well, cheaply, on a small model.

Often cheaper and more reliable than stuffing everything into a giant prompt, or paying for an even bigger model. **It lets a small model punch above its weight.**

LoRA and QLoRA

Parameter-efficient fine-tuning: freeze the giant model, train a tiny adapter.

LoRA Low-Rank Adaptation

- Inject small low-rank matrices into chosen layers
- Original weights stay frozen, only adapters train
- Trainable parameters drop by orders of magnitude

QLoRA Quantized Low-Rank Adaptation

- Everything LoRA does, plus a 4-bit base model
- Cuts VRAM again, so bigger models fit on one GPU
- The usual answer to an out-of-memory error

~0.1%

of parameters trained

4-bit

base weights under QLoRA

Comparable

quality to full fine-tuning

Training methodology: SFT and GRPO

Two ways to tell a model it did well. One shows the answer, the other scores attempts.

SFT Supervised Fine-Tuning

- Uses labelled data: prompt in, correct answer out
- The model imitates the answers you supply

```
{"instruction": "Summarize in one line.",
  "input":  "<article text>",
  "output": "<summary>"}
```

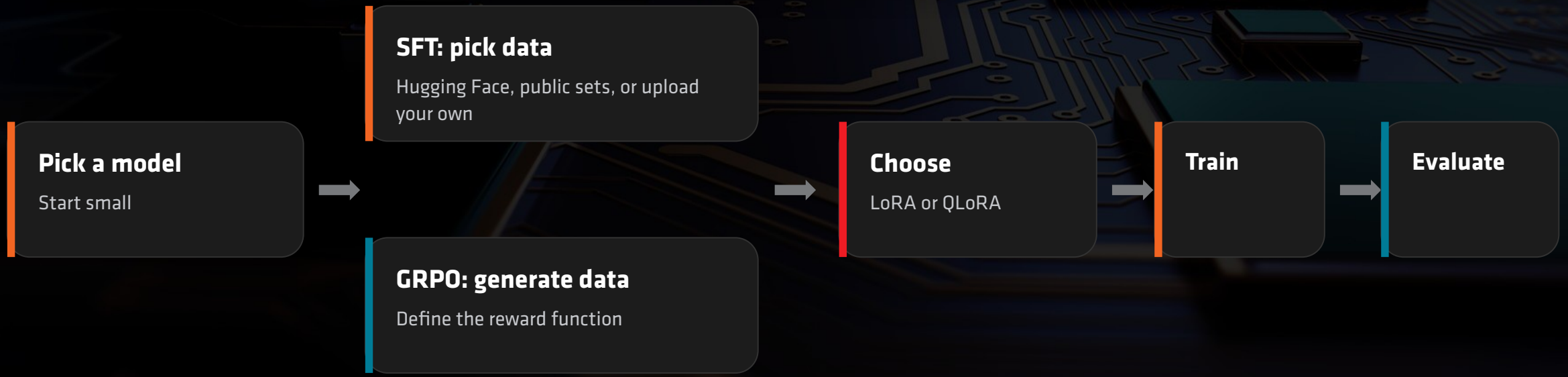
GRPO Group Relative Policy Optimization

- No labelled answer needed, only a way to score one
- Model generates a group of attempts and compares them

1. **Generate** several candidate answers
2. **Score** each with a reward function
3. **Prefer** the ones that scored above the group

The fine-tuning pipeline

Same spine either way. The data step is what differs.



Training knobs

`learning rate` `number of steps` `time limit` `max sequence length`

Unsloth

An open-source, no-code framework and UI for training, running and exporting LLMs **locally**, tuned for speed, memory efficiency and accessibility.

It sits as an optimized layer on top of Hugging Face Transformers, so your existing workflow still applies.

LoRA and QLoRA

built in, no glue code

Quantization

4-bit and 8-bit paths

Optimized kernels

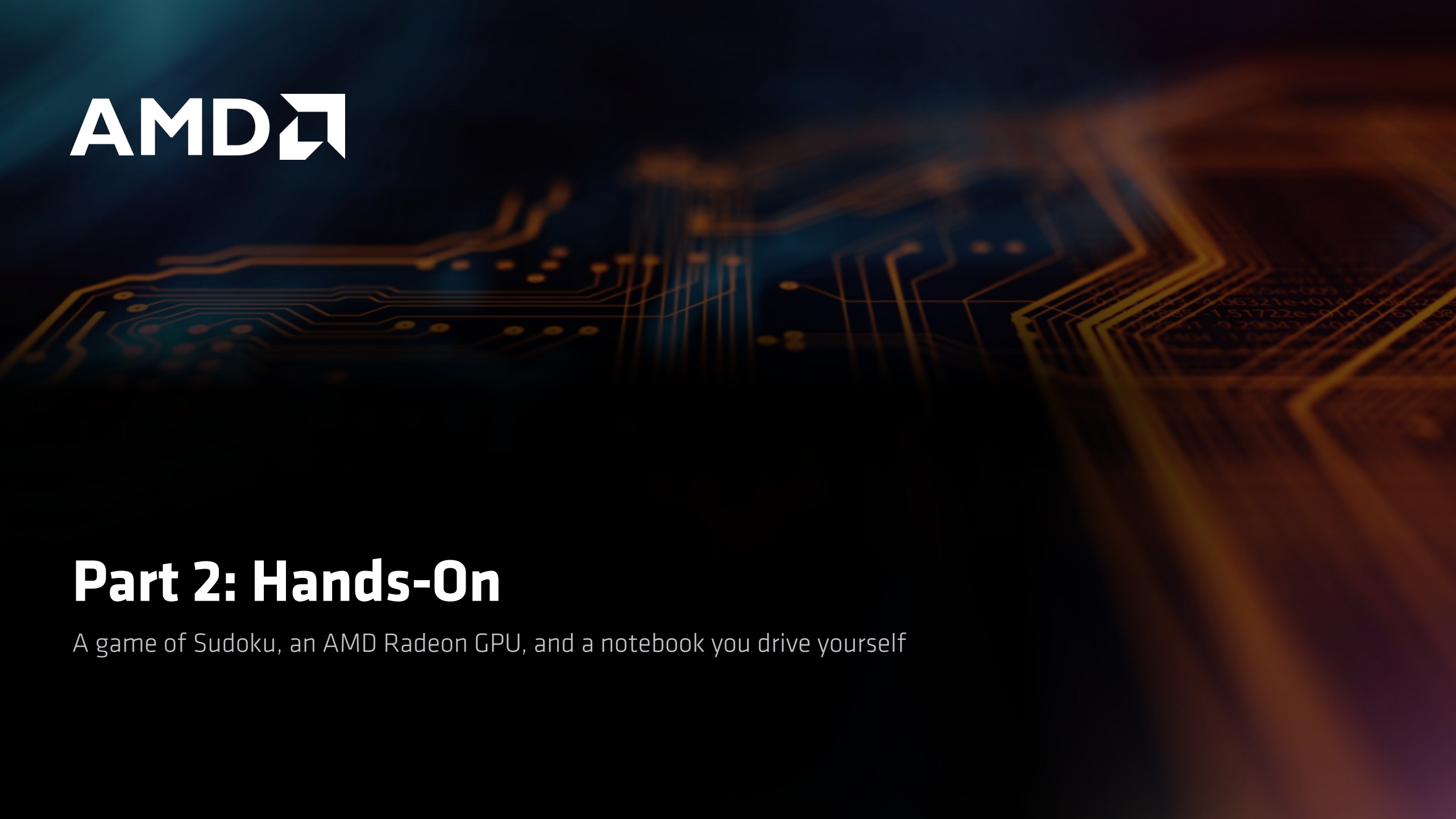
faster attention and MLP

Consumer hardware

one GPU is enough

In this workshop

Unsloth is what makes a GRPO run finish inside a session slot instead of overnight.



Part 2: Hands-On

A game of Sudoku, an AMD Radeon GPU, and a notebook you drive yourself

A game of Sudoku

Sudoku means **“single number”** in Japanese.

A logic puzzle: place digits 1 to 9 so every row, every column and every 3x3 box contains each digit exactly once.

Verifiable

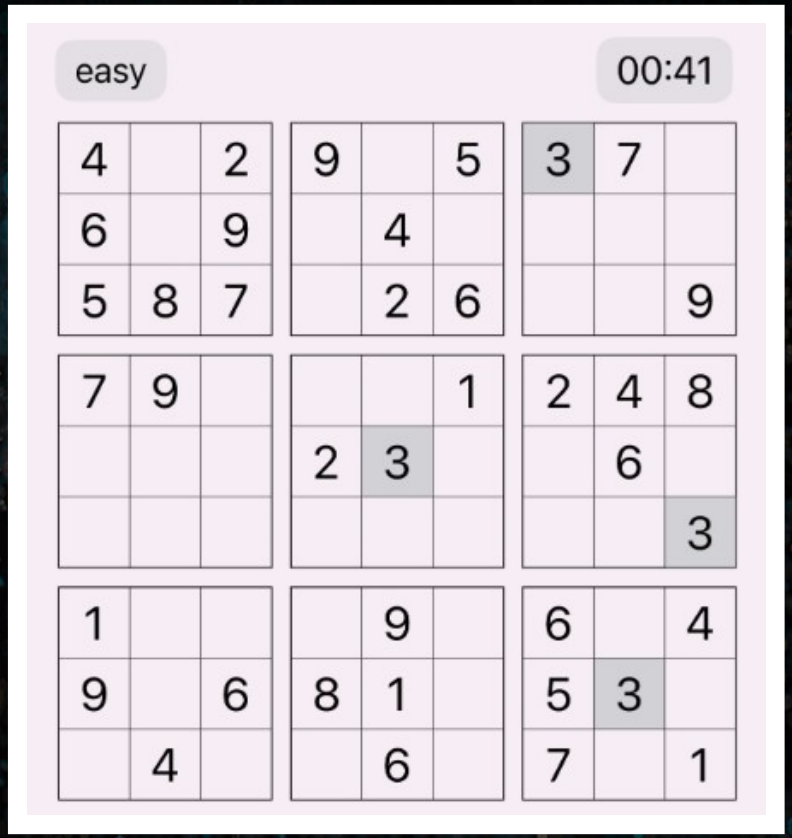
A board is right or wrong. No human judge needed.

Reward-friendly

Score partial credit per correct cell, row and box.

Genuinely hard

A general model gets it wrong. Improvement is visible.



Our training task: an easy grid, scored cell by cell

AMD Unsloth fine-tuning workshop

1 **Open the portal**

notebooks.amd.com/ind-dev

2 **Enter the password**

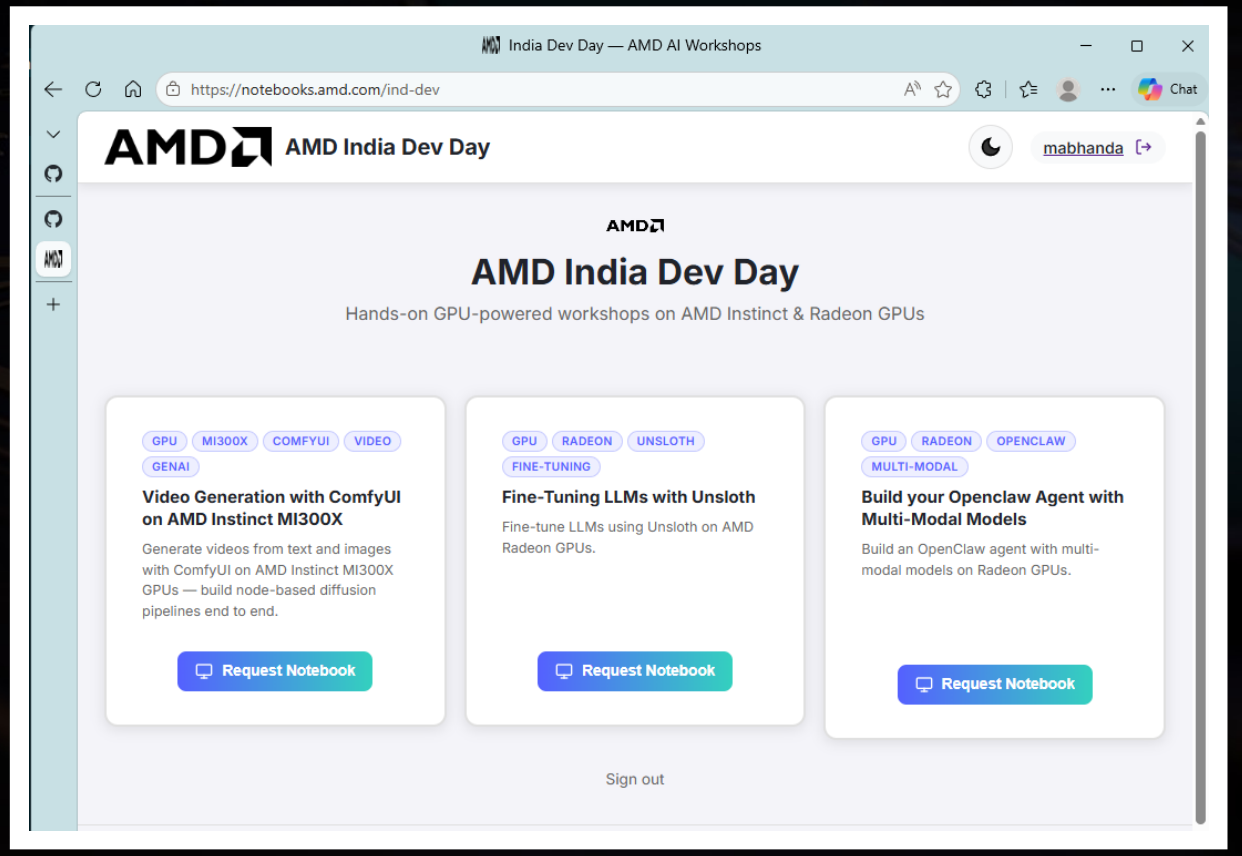
RadeonUnslothTuning

3 **Pick the Unsloth card**

Lands you on an AMD Radeon GPU

It is a real GPU, not a simulator

Hosted Jupyter, nothing to install locally



GPU visibility and health

Open a terminal inside Jupyter and confirm the GPU is really there before you train.

terminal

```
$ rocm-smi
$ rocminfo
```

rocm-smi

Live view: temperature, power, clocks, VRAM use

rocminfo

Static view: agents, ROCm version, GPU identity

GPU busy at 0% while a cell runs? It is not training.

```
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo# rocm-smi


========================================= ROCm System Management Interface =========================================
=
================================================== Concise Info ==================================================
=
Device  Node  IDs              Temp    Power  Partitions          SCLK  MCLK   Fan    Perf  PwrCap  VRAM%  GPU%
              (DID,     GUID)  (Edge)  (Avg)  (Mem, Compute, ID)
==================================================================================================================
=
0       3     0x744b,   49884  27.0°C  7.0W   N/A, N/A, 0         0Mhz  96Mhz  20.0%  auto  241.0W  0%     0%
==================================================================================================================
=
============================================== End of ROCm SMI Log ===============================================
=
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo#
```

```
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo# rocminfo
ROCk module version 6.16.13 is loaded
=====================
HSA System Attributes
=====================
Runtime Version:         1.18
Runtime Ext Version:     1.15
System Timestamp Freq.:  1000.000000MHz
Sig. Max Wait Duration:  18446744073709551615 (0xFFFFFFFFFFFFFFFF) (timestamp count)
Machine Model:           LARGE
System Endianness:       LITTLE
Mwaitx:                  DISABLED
XNACK enabled:           NO
DMAbuf Support:          YES
VMM Support:             YES

==========
HSA Agents
==========
*******
Agent 1
*******
  Name:                    AMD EPYC 9334 32-Core Processor
  Uuid:                    CPU-XX
  Marketing Name:          AMD EPYC 9334 32-Core Processor
  Vendor Name:             CPU
  Feature:                 None specified
  Profile:                 FULL PROFILE
```

rocminfo confirms the ROCm runtime version and every agent the host exposes, GPU and CPU alike.

Driving the notebook

Run cells top to bottom. The first model load is the slow one.

Watch the marker

`In [*]` means still running, `In [7]` means done

Do not skip ahead

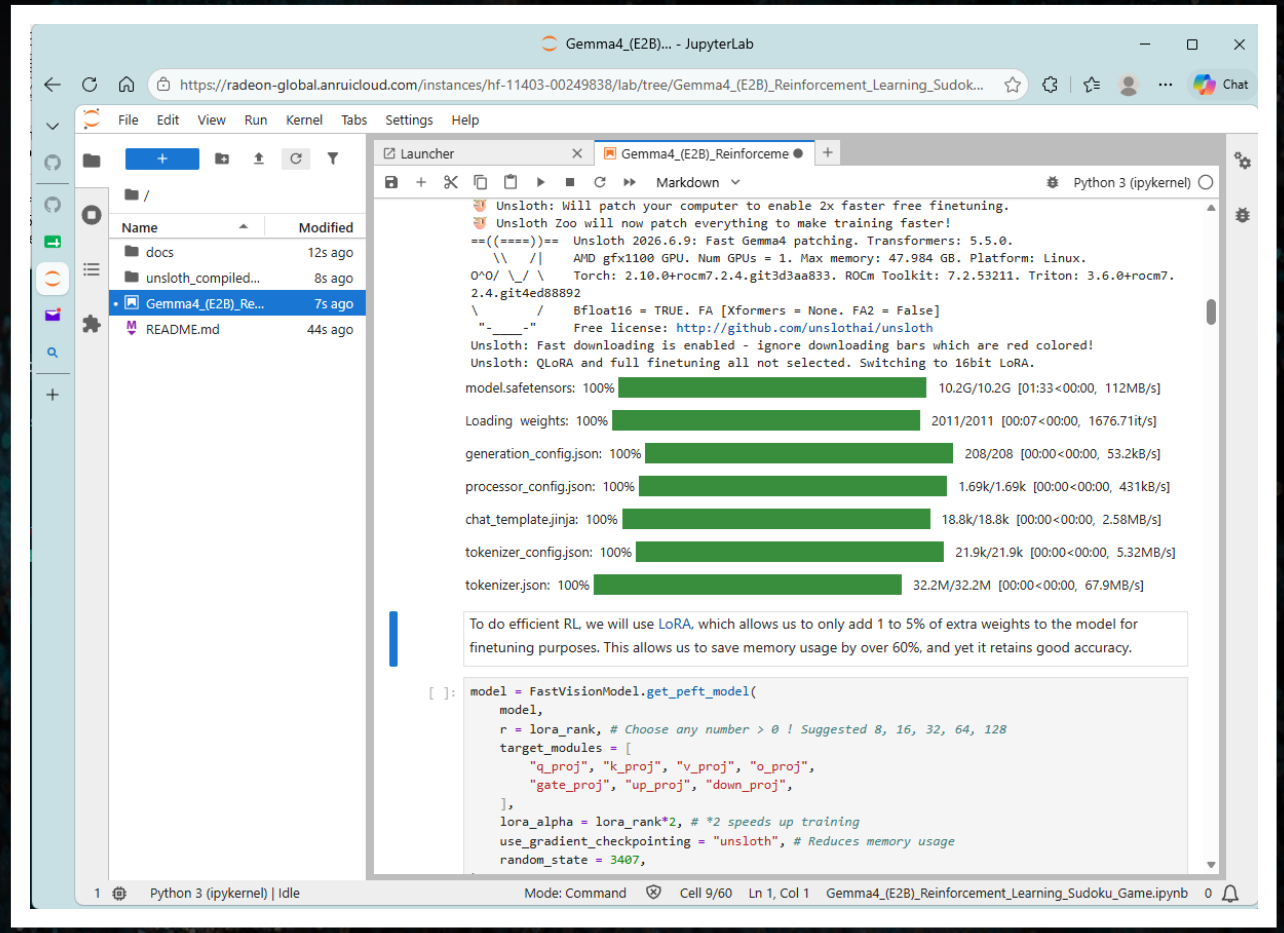
Later cells depend on variables the earlier ones define

Keep an eye on the GPU

Second terminal tab running `rocm-smi`

Cold start is normal

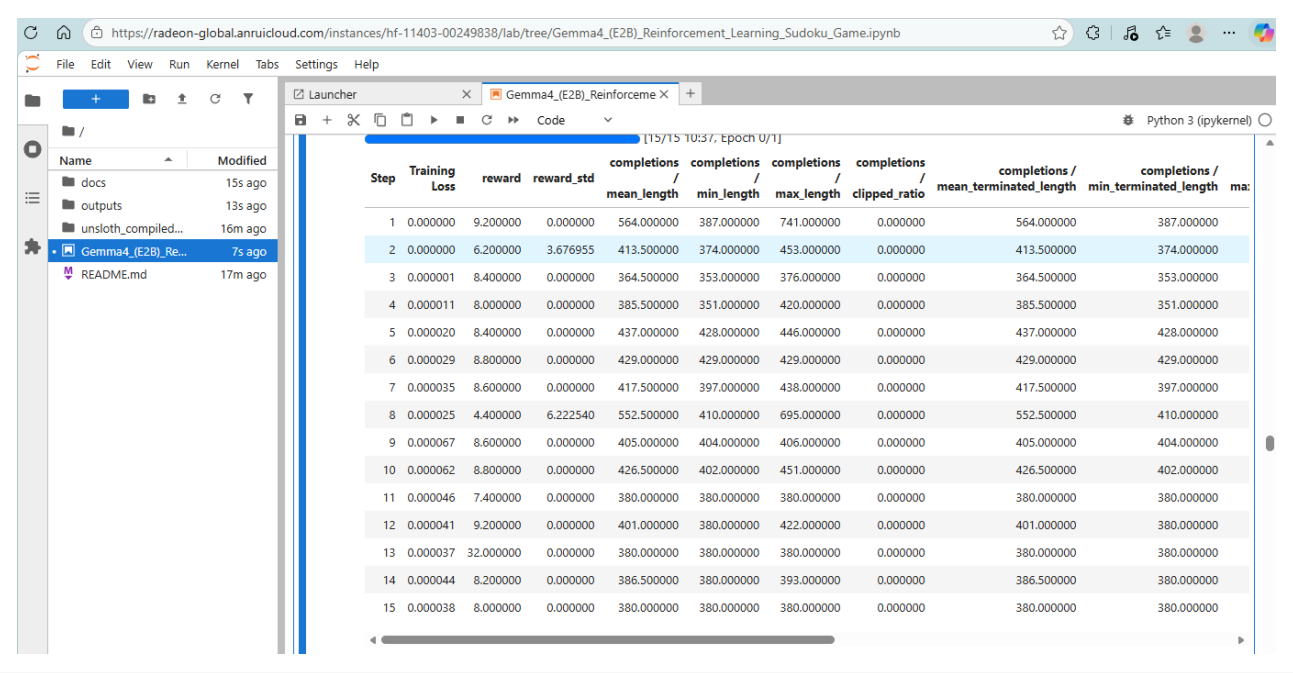
First load pulls weights. Retry a timed-out chat, the warm model is fast



Fine-tuning complete

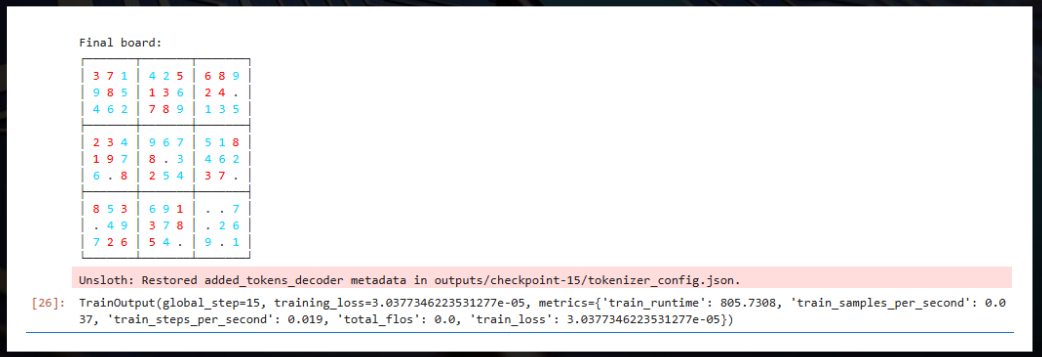
Fifteen GRPO steps on a Radeon GPU, and the reward curve moves.

DEMO: 15 STEPS



Step	Training Loss	reward	reward_std	completions / mean_length	completions / min_length	completions / max_length	completions / clipped_ratio	completions / mean_terminated_length	completions / min_terminated_length	ma:
1	0.000000	9.200000	0.000000	564.000000	387.000000	741.000000	0.000000	564.000000	387.000000	
2	0.000000	6.200000	3.676955	413.500000	374.000000	453.000000	0.000000	413.500000	374.000000	
3	0.000001	8.400000	0.000000	364.500000	353.000000	376.000000	0.000000	364.500000	353.000000	
4	0.000011	8.000000	0.000000	385.500000	351.000000	420.000000	0.000000	385.500000	351.000000	
5	0.000020	8.400000	0.000000	437.000000	428.000000	446.000000	0.000000	437.000000	428.000000	
6	0.000029	8.800000	0.000000	429.000000	429.000000	429.000000	0.000000	429.000000	429.000000	
7	0.000035	8.600000	0.000000	417.500000	397.000000	438.000000	0.000000	417.500000	397.000000	
8	0.000025	4.400000	6.222540	552.500000	410.000000	695.000000	0.000000	552.500000	410.000000	
9	0.000067	8.600000	0.000000	405.000000	404.000000	406.000000	0.000000	405.000000	404.000000	
10	0.000062	8.800000	0.000000	426.500000	402.000000	451.000000	0.000000	426.500000	402.000000	
11	0.000046	7.400000	0.000000	380.000000	380.000000	380.000000	0.000000	380.000000	380.000000	
12	0.000041	9.200000	0.000000	401.000000	380.000000	422.000000	0.000000	401.000000	380.000000	
13	0.000037	32.000000	0.000000	380.000000	380.000000	380.000000	0.000000	380.000000	380.000000	
14	0.000044	8.200000	0.000000	386.500000	380.000000	393.000000	0.000000	386.500000	380.000000	
15	0.000038	8.000000	0.000000	380.000000	380.000000	380.000000	0.000000	380.000000	380.000000	

Per-step training loss and reward, straight from the notebook



Final board produced by the tuned model

What to look for

- Reward trending up, not the absolute value
- A structurally valid board, no repeated digits

Troubleshooting

SYMPTOM	FIX
First chat response times out	Cold model load. Retry, the warm model answers fast.
Training data format issues	Check the Alpaca fields: instruction, input, output.
Out of memory during training	Smaller model, enable QLoRA 4-bit, lower batch size or sequence length.
Loss not decreasing	Raise the learning rate a little, train more steps, re-check the dataset format.
Cell never completes	Run rocm-smi. GPU use drops to zero when training is actually finished.
Need a first experiment idea?	Start with SFT on a Hugging Face set, then GRPO on math where answers are checkable.

Learn more

Unsloth	github.com/unslothai/unsloth	The framework itself
Unsloth docs	docs.unsloth.ai	Guides, notebooks, supported models
LoRA paper	arxiv.org/abs/2106.09685	Low-Rank Adaptation, Hu et al.
QLoRA paper	arxiv.org/abs/2305.14314	4-bit finetuning, Dettmers et al.
GRPO / DeepSeekMath	arxiv.org/abs/2402.03300	Where GRPO is introduced
Workshop notebook	notebooks.amd.com/ind-dev	Password: RadeonUnslothTuning


AMD